# Recomendador de cursos por intereses

## SQL Server 2025 y RAG local

Mini proyecto DP-800 | Nivel 3 avanzado | 26 de octubre de 2026

### Objetivo y caso de uso

Convertir una necesidad de aprendizaje expresada en español en cursos relevantes, respetando nivel, duración y temática. El sistema explica sus recomendaciones usando únicamente los cursos recuperados del catálogo.

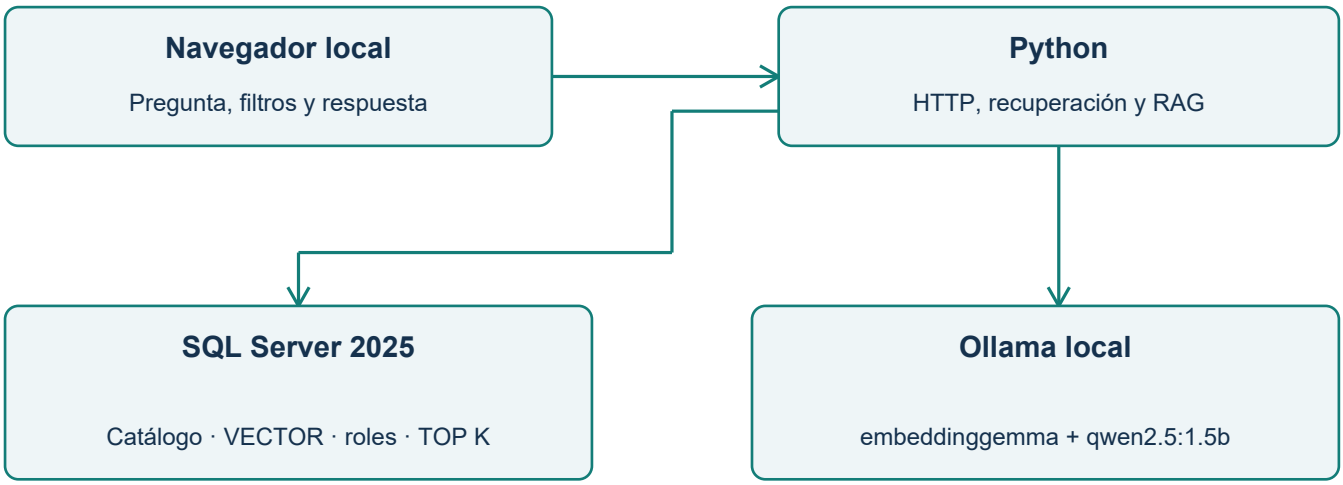
Ejemplo: «Quiero aprender a analizar ventas con SQL y tengo conocimientos básicos». La aplicación compara embeddings, aplica filtros SQL y presenta una orientación con referencias verificables.

### Alcance

131 cursos ficticios, diez categorías y dos modelos locales. Se incluyen diseño relacional, programación T-SQL, seguridad, optimización medida, búsqueda semántica y RAG. No se utilizan servicios de inferencia de pago.

El catálogo y la respuesta son una demostración académica; no representan una oferta real de cursos.

# Arquitectura y ejecución local




Carga inicial: Python genera embeddings con Ollama y los guarda en SQL Server.

Consulta: SQL devuelve fuentes; Python envía ese contexto al LLM local.

## Responsabilidad de cada componente

El navegador envía la petición a Python. embeddinggemma transforma el texto en un vector. El procedimiento SQL calcula distancias y devuelve cursos. Python forma el contexto para qwen2.5:1.5b, que genera motivos de recomendación. Los datos estructurados y las citas se completan desde SQL, no se redactan libremente.

| Componente | Tecnología / función |
|---|---|
| Datos | SQL Server 2025 local; tablas, vistas, procedimientos, JSON y VECTOR(768) |
| Búsqueda | Ollama embeddinggemma y distancia coseno en T-SQL |
| Generación | Ollama qwen2.5:1.5b; salida JSON estructurada |
| Interfaz | Python estándar, pyodbc, HTML/CSS/JavaScript |
| Herramientas | Windows 11, SSMS 22, VS Code, Git y repositorio GitHub privado |

Los modelos se descargan una vez. La ejecución consume recursos locales y genera/procesa tokens, pero no hay facturación por API. Cambiar el modelo de embeddings exige regenerar todo el espacio vectorial.

# Modelo de datos y programación SQL

| Objeto | Contenido y relación |
|---|---|
| dbo.Courses | CourseId PK; título, descripción, nivel, duración, atributos JSON, embedding, modelo y fecha. |
| dbo.Categories | CategoryId PK y nombre UNIQUE. Diez temáticas. |
| dbo.CourseCategories | PK compuesta y dos FK: relación muchos a muchos entre cursos y categorías. |
| dbo.vCourseCatalog / vCategories | Vistas de lectura que no exponen el vector al lector. |
| dbo.usp_RecommendCourses | Validación de parámetros, conversión de vector, filtros, CTE TOP K y agregado de categorías. |

## Integridad e índices

CHECK limita nivel y duración positiva; ISJSON valida los atributos adicionales. PK y UNIQUE crean los índices básicos. Las FK impiden enlaces a cursos o categorías inexistentes. La carga usa comprobaciones de existencia para evitar duplicados al repetirse.

## T-SQL avanzado

Se usan CTE para separar puntuación y selección, EXISTS correlacionado para filtrar temática, STRING_AGG para categorías, JSON_VALUE para atributos y ROW_NUMBER para la ampliación sintética. El procedimiento admite hasta diez resultados y comprueba los parámetros antes de consultar.

## Datos de prueba

El catálogo inicial tenía 31 cursos. Se añadieron 100 ejemplos sintéticos marcados con source=synthetic_scale_test. Los 131 registros tienen embeddings. Los datos adicionales no representan una oferta comercial real.

# Embeddings y búsqueda semántica

El texto de documento combina Title, Description y las listas skills y prerequisites de AdditionalAttributes. Se utiliza el formato «title: ... | text: ...». La consulta usa «task: search result | query: ...». Ambos se generan con embeddinggemma, 768 dimensiones, y el vector se almacena en SQL Server.

La distancia coseno compara la orientación de los vectores: 1 - producto escalar / producto de normas. Una distancia menor indica mayor afinidad. El ranking es una búsqueda exacta con filtros relacionales; no se necesita un índice aproximado para este catálogo.

El umbral 0,70 se eligió de forma empírica y se verificó con casos positivos y una consulta fuera de temática. No es una garantía universal de relevancia y debe reevaluarse con más ejemplos.

| Prueba real | Resultados |
|---|---|
| Búsqueda semántica | 10 |
| Principiante | 10 |
| Avanzado | 10 |
| Duración máxima | 3 |
| Temática SQL | 10 |
| Sin afinidad | 0 |
| Filtros combinados | 10 |
| Paráfrasis | 4 |

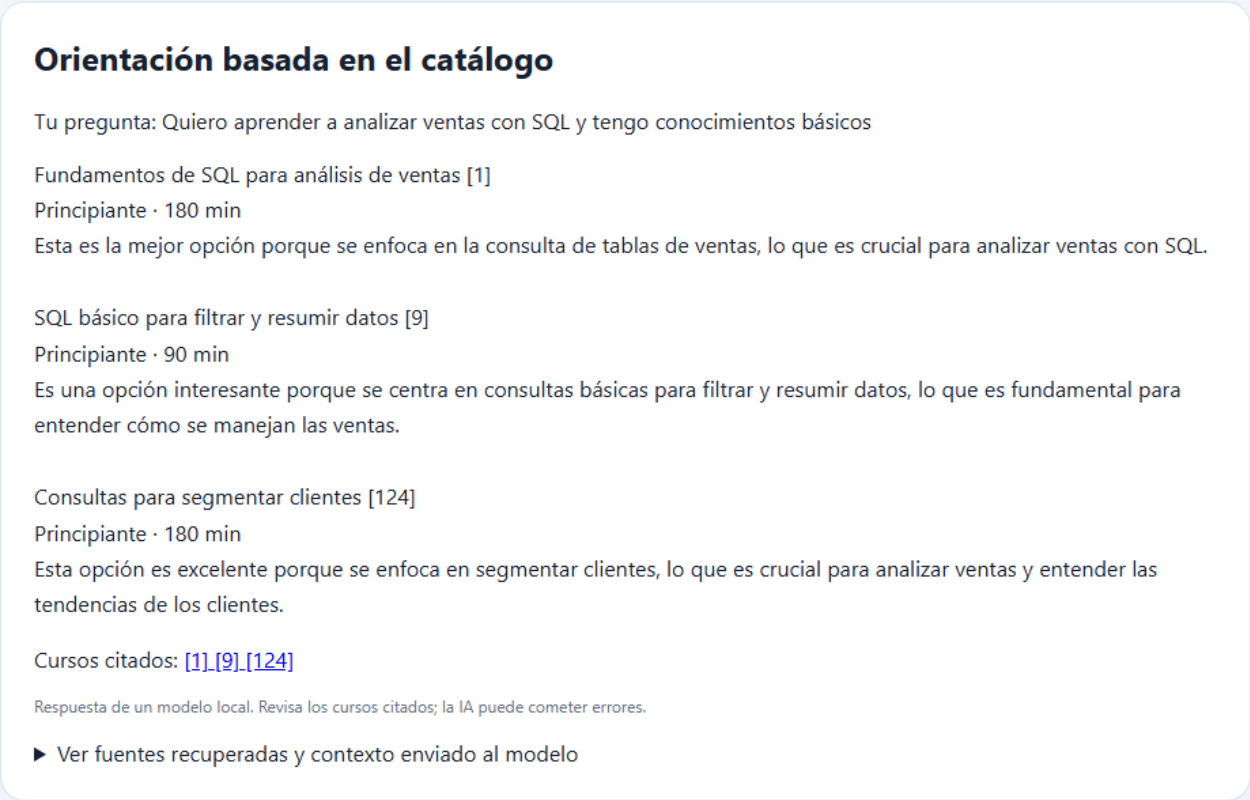
## Paráfrasis

Necesito convertir cifras comerciales en un tablero interactivo con gráficos

Primeros cursos recuperados: Indicadores de conversión comercial; Power BI: primeros informes de ventas.

# RAG: contexto, generación y fuentes

RAG añade recuperación de datos antes de la generación. El LLM recibe la pregunta y los cuatro cursos mejor clasificados, no el catálogo completo. Devuelve identificadores y motivos breves; Python valida las fuentes y muestra título, nivel y duración desde los datos recuperados.




Captura de una ejecución real de la interfaz local. El detalle desplegable permite examinar el contexto enviado al modelo.

## Abstención y límites

Sin cursos pertinentes no se invoca al modelo. Si se pregunta por precios, se informa de que el esquema no tiene esa información. Las fuentes se validan contra los cursos recuperados. La explicación libre todavía puede contener interpretaciones incorrectas: las citas no constituyen una prueba completa de veracidad.

Se probaron salida con identificadores desconocidos, referencias inconsistentes, datos cuantitativos no permitidos en el motivo y ausencia de fuentes. Las respuestas y el contexto reales se conservan en validation.json y ui-rag.json.

# Seguridad y permisos diferenciados

La demostración combina usuarios/roles de base de datos y permisos explícitos GRANT/DENY. CourseReaderDemo y CourseEditorDemo se crean WITHOUT LOGIN: no hace falta guardar contraseñas. Se verifican los permisos con EXECUTE AS USER y REVERT.

| Operación | Lector | Editor |
|---|---|---|
| Consultar vista del catálogo | Permitido | Permitido |
| Ejecutar recomendación | Permitido | No concedido |
| Leer directamente Courses y sus vectores | Denegado | Permitido |
| Actualizar Courses | Denegado | Permitido |
| Borrar Courses | Denegado | Denegado |

## Evidencia de funcionamiento

La prueba de seguridad confirma lectura de la vista, error 229 al acceder a la tabla base como lector, error 229 al modificarla y error 229 al borrar como editor. La actualización de prueba del editor se ejecuta dentro de una transacción que termina en ROLLBACK.

## Identidad efectiva de la aplicación

Python usa la identidad lectora para cada consulta y revierte antes de devolver la conexión al pool. La cadena de propiedad permite ejecutar el procedimiento sin conceder lectura directa de la tabla. Los valores de usuario se enlazan como parámetros, no se concatenan en SQL.

La cuenta de Windows de la demostración necesita permisos de instalación e impersonación. Para un servicio real debe usarse un login dedicado asociado al rol lector y retirar esos permisos administrativos. Las credenciales no se almacenan en el repositorio.

# Optimización: comparación controlada

La consulta inicial agregaba categorías de los candidatos antes de limitar el ranking. La versión final limita TOP K dentro de una CTE antes del agregado. Así reduce accesos a las tablas de categorías. Se comparan los mismos datos, vector, filtros y columnas de salida.

| Métrica por llamada | Inicial | CTE TOP K |
|---|---|---|
| Duración SQL media (ms) | 1.475 | 1.164 |
| CPU SQL media (ms) | 1.256 | 0.899 |
| Lecturas lógicas | 1145.0 | 331.0 |
| Mediana cliente ODBC (ms) | 2.641 | 2.616 |

Método: tres calentamientos y treinta llamadas pareadas, alternando el orden, sin limpiar cachés. Se comprueba igualdad de las filas completas. La generación de embeddings queda fuera de la medición. Query Store se filtra por object_id y se toman deltas antes/después, sumando las sentencias internas por llamada.

Las lecturas se reducen aproximadamente un 71 %. La mediana observada en el cliente cambia muy poco: el ruido y el coste de ODBC dominan a esta escala. La mejora de CPU y de lecturas no permite extrapolar latencia para otros equipos o catálogos.

## Propuesta descartada

Una primera variante con tabla temporal reducía lecturas, pero aumentaba la duración y la CPU. La CTE evita ese coste. Ambas mediciones se conservan como evidencia de evaluación y corrección del código asistido por IA.

# Verificación, problemas y conclusiones

Se ejecutaron cuatro pruebas unitarias y ocho escenarios de recuperación, además de los casos RAG positivo, sin datos y sin cursos pertinentes. Se verificó la identidad lectora y se reprodujo la instalación en CourseRecommender_Repro, con 131 embeddings y pruebas satisfactorias.

| Problema encontrado | Resolución |
|---|---|
| ODBC ntext y VECTOR | Conversión explícita de JSON a nvarchar(max) y después a VECTOR(768). |
| Alias SQL reservado y diagnósticos en Query Store | Alias corregido e identificación por object_id del procedimiento. |
| Citas y duración inventada por el LLM pequeño | Salida estructurada con IDs y motivos; datos cuantitativos y referencias compuestos por la aplicación desde SQL. |
| Mejora de lecturas sin mejora de tiempo | Medir la propuesta, conservar la evidencia y sustituir tabla temporal por CTE. |

## Conclusiones

El valor de la IA depende de la calidad de los datos, la recuperación y la validación. SQL Server conserva la integridad y los permisos; el modelo local redacta la orientación a partir del contexto. Las pruebas y métricas permiten explicar tanto el funcionamiento como sus límites.

Mejoras futuras: evaluación semántica con más consultas etiquetadas, verificación de afirmaciones generadas, cuenta de servicio dedicada y pruebas de carga con mayor concurrencia. El proyecto es una demostración académica, no una plataforma productiva.

## Referencias y reproducción

Código, scripts y evidencias: repositorio GitHub privado del proyecto (acceso por invitación). README contiene instalación y comandos. Fuentes oficiales: https://ollama.com/library/qwen2.5:1.5b · https://ollama.com/library/embeddinggemma · https://docs.ollama.com/api/chat · https://docs.ollama.com/capabilities/structured-outputs